Scripting Languages - Lecture 01

Lecture overview

This overview brings together the lecture's central ideas and the relationships between its key concepts.

This course adopts a hands-on scripting approach where ease of development and automating specific tasks, such as text processing for digital humanities, are prioritized over program efficiency. Code-along notebooks are published before each class, and every lecture except the first includes a practical session on Wednesdays from 4pm to 6pm.

While specific formats and rules regarding AI tool usage will be announced during the semester, the professor emphasizes that understanding code is essential to effectively control and take responsibility for any AI-generated solutions. Algorithms consistently identify patterns in large corpora without fatigue, allowing researchers to distribute their scripts so others can verify results or build upon them.

While some phrases like "the man reads the newspaper" are both syntactically valid and meaningful, others like "the lightbulb sings a banana" follow grammatical structure yet remain nonsensical due to illogical subject-verb-object relationships. Students must install Python and a Jupyter distribution on their own laptops for next week's practical sessions, as running code locally is preferred over using Google Colab, which executes on remote servers.

Slide 1: faculty of arts

This introductory lecture on scripting languages is presented by Tim Van de Cruys for the Faculty of Arts. The session, identified as Lecture 1, took place on Wednesday, 23 September 2026. No specific technical content or definitions were provided in the aligned source material for this title slide.

Slide 2: Introduction

This introductory course teaches programming using Python to collect, process, and analyze humanities data such as text, tables, and web sources. It assumes no prior experience while fostering an algorithmic way of thinking applicable across languages. The primary objective is practical application: writing scripts to effectively get things done in research contexts.

Slide 3: Scripting languages?

Scripting languages differ from traditional programming languages by executing code on the fly without prior compilation, though this distinction is diminishing due to modern hardware speed. This course adopts a hands-on scripting approach where ease of development and automating specific tasks, such as text processing for digital humanities, are prioritized over program efficiency. The primary goal is to quickly process large-scale data and automate repetitive tasks that would be tedious to perform manually.

Slide 4: Scripting languages = Python

The course primarily focuses on learning Python, a scripting language chosen for its ease and intuitive nature. Its code is designed to be easily understandable, resembling plain English. Additionally, Python offers access to many useful libraries, particularly those beneficial for data science applications.

Slide 5: Python

Slide 5 introduces the topic of Python within the context of scripting languages, dated May 28th. The slide attributes the content to tim.vandecruys@kuleuven.be and cites a Stack Overflow blog post regarding Python's incredible growth as its source. No additional explanatory details or professor commentary are provided for this specific section in the available materials.

Slide 6: A simple Python program:

Slide 6 introduces a basic Python program using the statement `print('Hello, World!')`. This example serves as a fundamental demonstration within the context of scripting languages. The slide is part of a lecture series dated 6/28 by tim.vandecruys@kuleuven.be.

Slide 7: A simple Python program:

Slide 7 contrasts a simple Python program, `print('Hello, World!')`, with its more verbose Java equivalent requiring class definitions and boilerplate code. The professor explains that Python is easier to interpret and read because it requires significantly less code to achieve the same result. This simplicity and intuitive grammar make Python a preferred choice for many developers compared to languages like Java.

Slide 8: Who is in the room?

This course accommodates students from diverse backgrounds, including Digital Humanities and Sociology (6 ECTS) or Artificial Intelligence and Statistics (4 ECTS). Lectures begin at a zero-experience level for beginners, while those with prior programming skills can access more challenging exercises in practical sessions. Both versions share the same lectures and exam, though the 6 ECTS track includes an additional project component.

Slide 9: Learning goals

The course learning goals combine theoretical foundations in computational thinking and programming language concepts with a primary emphasis on practical Python skills. Students will develop competencies in text processing, data analysis, visualization, and data collection while mastering researcher workflows like command-line usage and script execution. Additionally, the curriculum focuses on reading, evaluating, and debugging code, including that generated by AI tools, supported by dedicated practical sessions for hands-on application.

Slide 10: Format of the course

The course consists of 12 on-campus lectures with recordings posted on Toledo at the end of the term. Code-along notebooks are published before each class, and every lecture except the first includes a practical session on Wednesdays from 4pm to 6pm. These sessions feature tiered exercises (warm-up, core, challenge) with student assistance available during the time and solutions released afterwards.

Slide 11: Practical information

Course materials, including slides, notebooks, reading material, and exercises, are available online via the Toledo platform. Students can ask questions through the course forum on Toledo or during Q&A sessions in lectures and practicals. It is essential to regularly check the Toledo course page for updates.

Slide 12: Content of the course

This course covers Python fundamentals including values, variables, strings, lists, conditions, loops, functions, and dictionaries. It progresses to file handling, command line usage, modules, error management, object-oriented programming, text processing, regular expressions, and tabular data analysis with pandas. The curriculum concludes with web data collection, Digital Humanities applications, and coding with AI tools.

Slide 13: Evaluation

The 4 ECTS version is evaluated solely by a final exam, while the 6 ECTS version combines a final exam (two-thirds) with an independent project applying course techniques to a chosen dataset (one-third). The full project brief appears in lecture 5. Additionally, two ungraded assignments and a mock exam are strongly recommended for self-assessment but do not count toward the final grade.

Slide 14: The exam

The course concludes with a computer-based exam held on campus in January, where students submit programming exercises as notebooks. These tasks assess practical skills including writing, reading, explaining, and fixing code, mirroring the content of practical sessions. While specific formats and rules regarding AI tool usage will be announced during the semester, the professor emphasizes that understanding code is essential to effectively control and take responsibility for any AI-generated solutions.

Slide 15: Computers for DH research?

Computers enable Digital Humanities research by automating tedious tasks, amplifying analysis beyond human capacity, and ensuring reproducibility through shared data and code. Algorithms consistently identify patterns in large corpora without fatigue, allowing researchers to distribute their scripts so others can verify results or build upon them. This approach supports practical applications like text processing, exploratory data analysis, and collecting data from the internet.

Slide 16: Programming for DH research?

To process data automatically in Digital Humanities research, scholars can either use existing tools or create custom scripts. While existing tools offer immediate functionality, they are limited to predefined features and often make repeating action sequences difficult. Conversely, writing one's own programs requires a higher initial investment but provides endless opportunities for automation and ensures that research results are reproducible.

Slide 17: Learning to program in 2026

Although large language models can generate Python code, learning programming remains essential to specify needs precisely, judge correctness, and debug failures on specific data. Students must understand the underlying logic to take responsibility for results, as generated code often appears correct but contains subtle errors. Consequently, the advice for the first weeks is to type code manually while using AI tools solely for explanation rather than solution generation.

Slide 18: Python is a language

Python is a scripting language characterized by three core components: words, syntax, and semantics. Words are the primitive constructs or basic building blocks used to form larger units. Syntax defines the rules stipulating how these words combine, while semantics represents the meaning derived from both the words and their specific combination according to syntactic rules.

Slide 19: Words

Slide 19 is titled "Words" and belongs to the "Scripting languages" course (English, slide 18/28) by Tim Vandecruys. The slide contains only this title and contact information, with no accompanying definitions, examples, or technical content provided in the source material. Consequently, no specific concepts or explanations regarding the topic of words can be derived from this section alone.

Slide 20: Words

Slide 20 is titled "Words" and identifies the course context as "Scripting languages," specifically focusing on Python. The slide indicates this is the 18th of 28 sessions and lists the instructor's contact email as tim.vandecruys@kuleuven.be. No additional definitions, examples, or explanatory content regarding the topic of words in Python are provided in the source material for this section.

Slide 21: Words

In the context of scripting languages like Python, "words" encompass not only textual identifiers but also numerical values and simple arithmetic operators such as addition (+), subtraction (-), and multiplication (*). This slide establishes that these fundamental elements are treated as basic units or tokens within the language's structure.

Slide 22: Words

Python programming relies on a limited set of English words known as keywords, such as "while," alongside numbers and simple operators like plus, minus, and multiplication. Additionally, Python allows users to invent their own custom words called variables, which hold specific meaning defined by the programmer. Together, these elements constitute the fundamental vocabulary used to construct scripts in this language.

Slide 23: Syntax

In the context of scripting languages, syntax refers to the rules that determine whether an expression is structurally correct. Some expressions are syntactically valid, meaning they adhere to these rules, while others are not. This distinction establishes the fundamental requirement for code to be recognized as properly formed before it can be executed or evaluated.

Slide 24: Syntax

Syntax defines the rules determining whether expressions are valid within a language. The slide illustrates this using English: "the dog chases the cat" is syntactically valid because it correctly arranges a subject, verb, and object. In contrast, jumbled phrases like "cat library the sleep bottle" violate these structural rules and are therefore invalid.

Slide 25: Syntax

Syntax in English distinguishes between expressions that are structurally valid and those that are not. For instance, "the dog chases the cat" is syntactically valid, whereas "cat library the sleep bottle" is not. This slide illustrates how word order determines whether a sequence of words forms a grammatically correct sentence.

Slide 26: Syntax

Syntax defines the structural rules a programming language requires for valid expressions. In Python, `2.5 * 14` is syntactically valid and executes successfully, whereas `"welcome" 2.5 in` triggers a syntax error because it violates these rules. Just as nonsensical word sequences like "cat library the sleep bottle" cannot be parsed in English, code must strictly follow syntactic conventions before its meaning, or semantics, can be evaluated.

Slide 27: Syntax

In Python, expressions must adhere to specific syntactic rules to be valid and interpretable by the language. The expression `2.5 * 14` is syntactically valid because it correctly combines numbers with a multiplication operator. Conversely, `"welcome" 2.5 in` is not syntactically valid as it lacks proper constituents and fails to follow Python's structural rules, preventing interpretation.

Slide 28: Semantics

Semantics concerns the meaning of expressions within a language. While certain expressions may be syntactically valid, they can still be non-sensical if they lack logical meaning. This distinction highlights that correct structure does not guarantee meaningful content in scripting languages.

Slide 29: Semantics

Semantics refers to the meaning of expressions constructed using syntactic rules. While some phrases like "the man reads the newspaper" are both syntactically valid and meaningful, others like "the lightbulb sings a banana" follow grammatical structure yet remain nonsensical due to illogical subject-verb-object relationships. This distinction between syntactic validity and semantic sense applies equally to English and Python programming languages.

Slide 30: Semantics

Semantics distinguishes between expressions that are syntactically valid yet non-sensical. The phrase "the man reads the newspaper" is semantically felicitous, whereas "the lightbulb sings a banana" is semantically anomalous despite being grammatically correct. This contrast illustrates how meaning constraints operate independently of syntactic structure in English.

Slide 31: Semantics

Semantics in Python distinguishes between syntactically valid expressions that are meaningful versus those that are nonsensical. While `2 + 5` is valid and executes correctly, `"welcome" + 1` adheres to syntax rules but fails semantically because Python cannot combine a string and an integer. This type mismatch triggers an error, illustrating that correct structure does not guarantee logical meaning or successful execution.

Slide 32: Semantics

Semantics distinguishes between expressions that are syntactically valid but non-sensical. In Python, the expression `2 + 5` is semantically felicitous because it makes logical sense, whereas `"welcome" + 1` is semantically anomalous despite being syntactically correct. This slide illustrates how certain operations fail to produce meaningful results even when they follow proper grammatical rules.

Slide 33: Semantics (bis)

The sentence "I saw the man with the telescope" illustrates semantic ambiguity in English, where the phrase "with the telescope" can modify either the noun (the man possesses it) or the verb (the observer uses it). While humans typically infer a single intended meaning, programming languages face similar challenges where expressions possess one specific meaning that may differ from the programmer's intent. This concept is exemplified by arithmetic expressions like "2 plus 5 times 10," highlighting the critical need for precise semantic definitions in code.

Slide 34: Semantics (bis)

In Python, every expression possesses exactly one single meaning. However, this definitive meaning may differ from what the programmer originally intended when writing the code. This distinction highlights that while the language's interpretation is unambiguous, it does not guarantee alignment with the developer's expectations.

Slide 35: Semantics (bis)

In Python, expressions possess a single, unambiguous meaning determined by strict evaluation rules, which may differ from a programmer's intent. For instance, `2 + 5 * 10` evaluates to 52 because multiplication precedes addition, whereas a human might incorrectly expect 70 by summing first. Without explicit brackets, expressions that appear ambiguous to humans are resolved definitively by the computer's precedence rules.

Slide 36: Semantics (bis)

In Python, expressions possess a single, unambiguous meaning defined by the language semantics. However, this precise meaning may differ from what the programmer originally intended when writing the code. The slide illustrates this concept with the expression `(2 + 5) * 10`, highlighting the distinction between syntactic correctness and semantic expectation in scripting languages.

Slide 37: Semantics (bis)

In Python, expressions possess a single, unambiguous meaning defined by the language rules. However, this precise semantic meaning may differ from what the programmer originally intended when writing the code. The slide illustrates this concept using the arithmetic expression "2 + (5 * 10)" to highlight how scripting languages interpret operations strictly according to their defined semantics.

Slide 38: Learning a language

Learning Python mirrors learning a spoken language, requiring practice and the acceptance of mistakes as essential for progress. Syntactic errors are common but easily identified by Python's explicit error messages, while significant semantic anomalies like type errors are also flagged. However, subtle semantic differences or ambiguity may produce no error message yet cause unexpected behaviors such as crashes, infinite loops, or incorrect results because the code's meaning differs from the programmer's intent.

Slide 39: Python script

A Python script is a sequence of definitions and commands written one after another. Definitions are evaluated by the language for later use, while commands (statements) instruct the interpreter to execute specific actions. These scripts can either be typed directly into a shell or stored in a file that is read into the shell for evaluation.

Slide 40: Objects

In Python, scripts manipulate data objects where each object's type defines the specific operations permissible on it, analogous to how a banana (fruit) can be eaten or Kim (human) can walk. Objects are categorized as either scalar, which cannot be subdivided like a whole number, or non-scalar, which possess an internal structure that allows access to individual elements, such as items within a list.

Slide 41: How to run Python code?

Running Python code involves three primary methods: scripting, interactive shells, and notebooks. The scripting approach follows a two-step process where you write and save code in a file, then execute it with a Python interpreter, repeating this cycle to solve errors. Alternatively, an interactive shell allows typing commands directly for immediate execution, while notebooks provide the most interactive experience by combining explanatory text, code, and results within a web browser interface like Google Colab.

Slide 42: Installing Python

The professor recommends installing the Anaconda distribution from anaconda.com rather than the main Python release, which can be cumbersome for beginners. Anaconda is a user-friendly option suitable for data science and machine learning that simplifies package management and includes Jupyter Notebook for practical sessions. The download page automatically detects your operating system to provide the correct installer for Windows, macOS, or Linux.

Slide 43: Reference material

The recommended reference material is "Python for Everybody" by Charles Severance, available at py4e.com. This resource focuses on Python for data processing and includes a book, additional videos, and exercises under a Creative Commons license. It serves as the primary supplementary text for this course module on scripting languages.

Slide 44: For next week

Students must install Python and a Jupyter distribution on their own laptops for next week's practical sessions, as running code locally is preferred over using Google Colab, which executes on remote servers. Additionally, prepare by reading Chapters 1 and 2 of the free online resource "Python for Everybody," which serves as a reference manual for basic Python concepts. Support for installation issues is available via the course forum, lectures, or practical sessions.